



Assignment 6

Metoder i Kunstig Intelligens

Aditi Ravi Deshpande

March 8, 2026

Contents

1	MLP	3
1.1	Parameters	3
1.2	Training	3
2	CNN	5
2.1	Parameters	5

3	Reflection	7
3.1	Comparison of the two models	7
3.2	Categorical Cross Entropy	7

1 MLP

1.1 Parameters

I added another layer with 32 units, and one more with 8. Both the layers had a ReLu activation.

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 28, 28, 16)	32
dense_1 (Dense)	(None, 28, 28, 32)	544
dense_2 (Dense)	(None, 28, 28, 8)	264
flatten (Flatten)	(None, 6272)	0
dense_3 (Dense)	(None, 10)	62,730

Total params: 63,570 (248.32 KB)
Trainable params: 63,570 (248.32 KB)
Non-trainable params: 0 (0.00 B)

Figure 1: Parameters of MLP model

1.2 Training

I have a learning rate of 0.01 and 50 epochs. Here is the evolution of the accuracy and loss:

```

Epoch 1/50
422/422 16s 36ms/step - accuracy: 0.7722 - loss: 1.0143 - val_accuracy: 0.9123 - val_loss: 0.3090
Epoch 2/50
422/422 15s 35ms/step - accuracy: 0.8980 - loss: 0.3517 - val_accuracy: 0.9222 - val_loss: 0.2669
Epoch 3/50
422/422 14s 34ms/step - accuracy: 0.9086 - loss: 0.3176 - val_accuracy: 0.9270 - val_loss: 0.2584
Epoch 4/50
422/422 14s 32ms/step - accuracy: 0.9140 - loss: 0.3033 - val_accuracy: 0.9310 - val_loss: 0.2452
Epoch 5/50
422/422 18s 27ms/step - accuracy: 0.9168 - loss: 0.2948 - val_accuracy: 0.9323 - val_loss: 0.2484
Epoch 6/50
422/422 14s 32ms/step - accuracy: 0.9192 - loss: 0.2890 - val_accuracy: 0.9328 - val_loss: 0.2431
Epoch 7/50
422/422 13s 31ms/step - accuracy: 0.9197 - loss: 0.2845 - val_accuracy: 0.9352 - val_loss: 0.2358
Epoch 8/50
422/422 13s 31ms/step - accuracy: 0.9216 - loss: 0.2807 - val_accuracy: 0.9342 - val_loss: 0.2371
Epoch 9/50
422/422 12s 29ms/step - accuracy: 0.9215 - loss: 0.2783 - val_accuracy: 0.9345 - val_loss: 0.2384
Epoch 10/50
422/422 12s 29ms/step - accuracy: 0.9223 - loss: 0.2750 - val_accuracy: 0.9362 - val_loss: 0.2331
Epoch 11/50
422/422 14s 33ms/step - accuracy: 0.9227 - loss: 0.2732 - val_accuracy: 0.9347 - val_loss: 0.2343
Epoch 12/50

```

Figure 2: MLP epochs 1-15 training

```

Epoch 40/50
422/422 14s 34ms/step - accuracy: 0.9310 - loss: 0.2467 - val_accuracy: 0.9385 - val_loss: 0.2344
Epoch 41/50
422/422 15s 36ms/step - accuracy: 0.9315 - loss: 0.2456 - val_accuracy: 0.9355 - val_loss: 0.2299
Epoch 42/50
422/422 15s 34ms/step - accuracy: 0.9307 - loss: 0.2454 - val_accuracy: 0.9355 - val_loss: 0.2315
Epoch 43/50
422/422 15s 36ms/step - accuracy: 0.9316 - loss: 0.2444 - val_accuracy: 0.9372 - val_loss: 0.2292
Epoch 44/50
422/422 16s 37ms/step - accuracy: 0.9321 - loss: 0.2443 - val_accuracy: 0.9385 - val_loss: 0.2277
Epoch 45/50
422/422 14s 33ms/step - accuracy: 0.9314 - loss: 0.2436 - val_accuracy: 0.9350 - val_loss: 0.2336
Epoch 46/50
422/422 15s 34ms/step - accuracy: 0.9330 - loss: 0.2422 - val_accuracy: 0.9377 - val_loss: 0.2279
Epoch 47/50
422/422 14s 33ms/step - accuracy: 0.9319 - loss: 0.2419 - val_accuracy: 0.9352 - val_loss: 0.2369
Epoch 48/50
422/422 15s 35ms/step - accuracy: 0.9320 - loss: 0.2415 - val_accuracy: 0.9360 - val_loss: 0.2301
Epoch 49/50
422/422 15s 36ms/step - accuracy: 0.9326 - loss: 0.2406 - val_accuracy: 0.9388 - val_loss: 0.2425
Epoch 50/50
422/422 15s 35ms/step - accuracy: 0.9324 - loss: 0.2403 - val_accuracy: 0.9388 - val_loss: 0.2247
013/313 3s 8ms/step - accuracy: 0.9271 - loss: 0.2653

```

Figure 3: Final result of training

The accuracy becomes 0.9271 , approximately 0.93 for the MLP model.

2 CNN

2.1 Parameters

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 8)	48
conv2d_1 (Conv2D)	(None, 28, 28, 32)	1,056
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 10)	62,730

Total params: 63,826 (249.32 KB)
Trainable params: 63,826 (249.32 KB)
Non-trainable params: 0 (0.00 B)

Figure 4: Parameters of CNN model

I added another convolutional layer with 32 extra filters and relu activation function with same padding. I have a learning rate of 0.01 and 50 epochs. Here is the evolution of the accuracy and loss:

```

Epoch 1/50
422/422 ————— 12s 28ms/step - accuracy: 0.7783 - loss: 0.9469 - val_accuracy: 0.9143 - val_loss: 0.3011
Epoch 2/50
422/422 ————— 11s 27ms/step - accuracy: 0.8967 - loss: 0.3548 - val_accuracy: 0.9250 - val_loss: 0.2638
Epoch 3/50
422/422 ————— 13s 31ms/step - accuracy: 0.9053 - loss: 0.3264 - val_accuracy: 0.9288 - val_loss: 0.2531
Epoch 4/50
422/422 ————— 9s 22ms/step - accuracy: 0.9094 - loss: 0.3113 - val_accuracy: 0.9293 - val_loss: 0.2442
Epoch 5/50
422/422 ————— 9s 22ms/step - accuracy: 0.9125 - loss: 0.3019 - val_accuracy: 0.9317 - val_loss: 0.2375
Epoch 6/50
422/422 ————— 14s 32ms/step - accuracy: 0.9146 - loss: 0.2936 - val_accuracy: 0.9342 - val_loss: 0.2345
Epoch 7/50
422/422 ————— 14s 33ms/step - accuracy: 0.9170 - loss: 0.2861 - val_accuracy: 0.9372 - val_loss: 0.2244
Epoch 8/50
422/422 ————— 13s 32ms/step - accuracy: 0.9206 - loss: 0.2775 - val_accuracy: 0.9358 - val_loss: 0.2215
Epoch 9/50
422/422 ————— 15s 34ms/step - accuracy: 0.9226 - loss: 0.2678 - val_accuracy: 0.9388 - val_loss: 0.2132
Epoch 10/50
422/422 ————— 13s 31ms/step - accuracy: 0.9249 - loss: 0.2594 - val_accuracy: 0.9423 - val_loss: 0.2073
Epoch 11/50
422/422 ————— 12s 29ms/step - accuracy: 0.9280 - loss: 0.2497 - val_accuracy: 0.9415 - val_loss: 0.2115
Epoch 12/50
422/422 ————— 10s 24ms/step - accuracy: 0.9319 - loss: 0.2379 - val_accuracy: 0.9480 - val_loss: 0.1887
Epoch 13/50
422/422 ————— 12s 27ms/step - accuracy: 0.9361 - loss: 0.2254 - val_accuracy: 0.9527 - val_loss: 0.1773

```

Figure 5: Cnn epochs 1-15 training

```

Epoch 45/50
422/422 ————— 11s 27ms/step - accuracy: 0.9836 - loss: 0.0559 - val_accuracy: 0.9818 - val_loss: 0.0671
422/422 ————— 11s 27ms/step - accuracy: 0.9836 - loss: 0.0559 - val_accuracy: 0.9818 - val_loss: 0.0671
Epoch 46/50
422/422 ————— 13s 32ms/step - accuracy: 0.9841 - loss: 0.0546 - val_accuracy: 0.9835 - val_loss: 0.0616
Epoch 47/50
422/422 ————— 11s 25ms/step - accuracy: 0.9846 - loss: 0.0537 - val_accuracy: 0.9838 - val_loss: 0.0620
Epoch 48/50
422/422 ————— 12s 28ms/step - accuracy: 0.9850 - loss: 0.0528 - val_accuracy: 0.9835 - val_loss: 0.0617
Epoch 49/50
422/422 ————— 10s 24ms/step - accuracy: 0.9845 - loss: 0.0522 - val_accuracy: 0.9828 - val_loss: 0.0619
Epoch 50/50
422/422 ————— 11s 26ms/step - accuracy: 0.9850 - loss: 0.0512 - val_accuracy: 0.9838 - val_loss: 0.0618
313/313 ————— 2s 6ms/step - accuracy: 0.9796 - loss: 0.0599

```

Figure 6: Final result of training

The accuracy becomes 0.9796 , approximately 0.98 for the

CNN model.

3 Reflection

3.1 Comparison of the two models

The CNN performs much better than MLP on MNIST, and it receives higher accuracy and faster convergence. This is because the convolutional layers in the CNN detect local patterns like edges and strokes, which are highly informative for digit recognition. It sends the same filter throughout the whole image, so it detects the same patterns several times as well. In contrast, the MLP treats each pixel independently, so it requires more parameters to learn similar features. Even when the MLP has a large number of parameters, it struggles to generalise as well as the CNN. In general, CNN parameter efficiency and its ability to capture local correlations make it better suited for image classification tasks like MNIST.

3.2 Categorical Cross Entropy

$$\mathcal{L}_{\text{avg}} = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_i^{(n)} \log(\hat{y}_i^{(n)})$$

For MNIST classification, the network's final layer is a softmax with 10 units, one for each digit (0–9). The cross-entropy categor-

ical loss measures how different the predicted probability distribution is from the true one-hot encoding. If the correct class has high probability, the loss is low. If the model assigns low probability to the correct class, the loss becomes large.

The softmax function converts the raw outputs into probabilities that sum to 1. Each unit represents the predicted probability of the model that the image belongs to that digit. True labels (one-hot encoding) The correct class is represented as a one-hot vector (for example, digit 3 $\rightarrow$ $[0,0,0,1,0,0,0,0,0]$).